harness

Is Resilience Testing Negotiable in SDLC?

Use Chaos Engineering to improve Resilience

Umasankar Mukkara

Head of Resilience Testing | Harness Inc

Reliability · Resilience · AI-Powered Risk Detection

1. Reliability vs Resilience
2. SREs & Chaos Engineering
3. Chaos in the AI Era

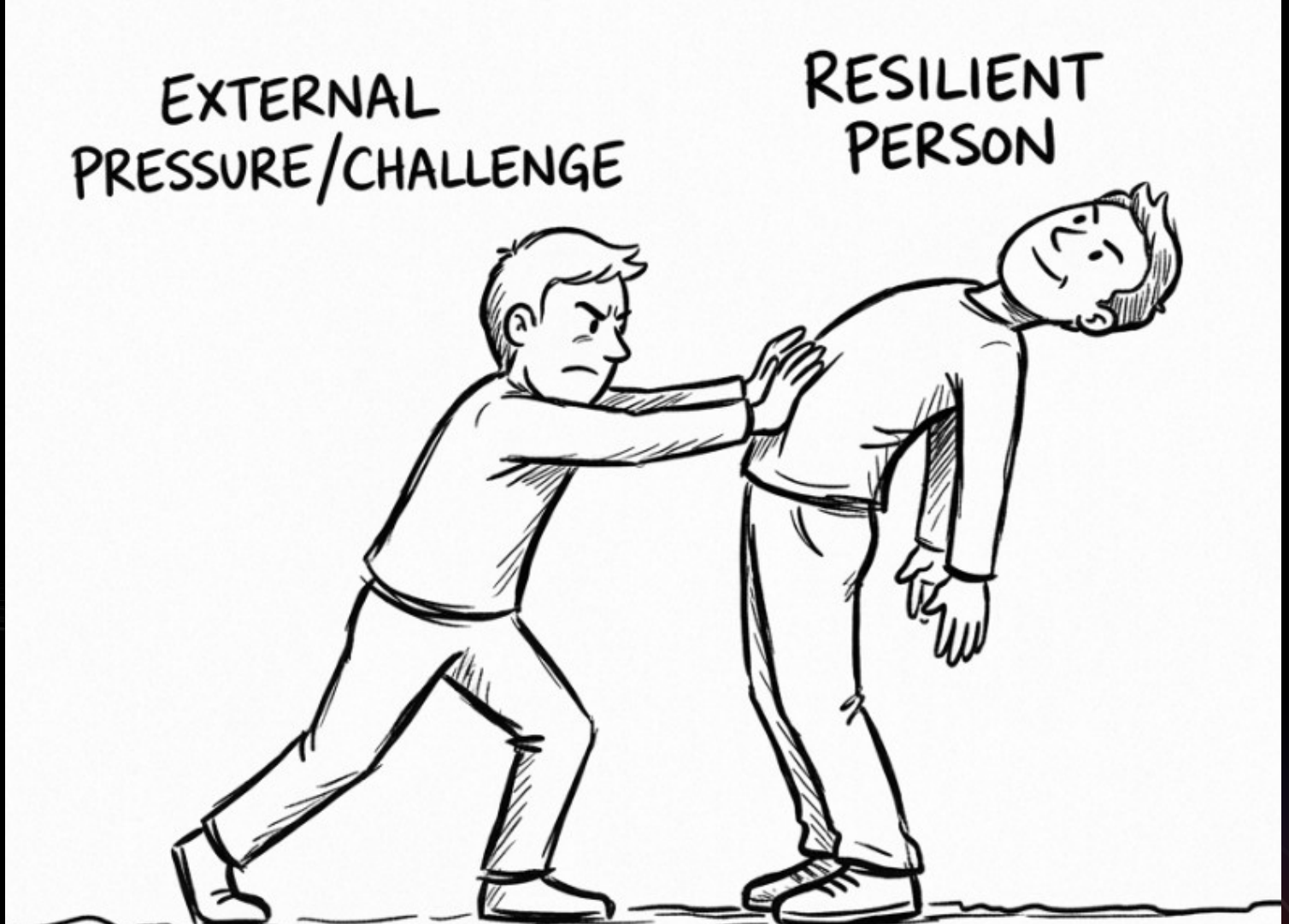


Resilience Testing

Customer Notes

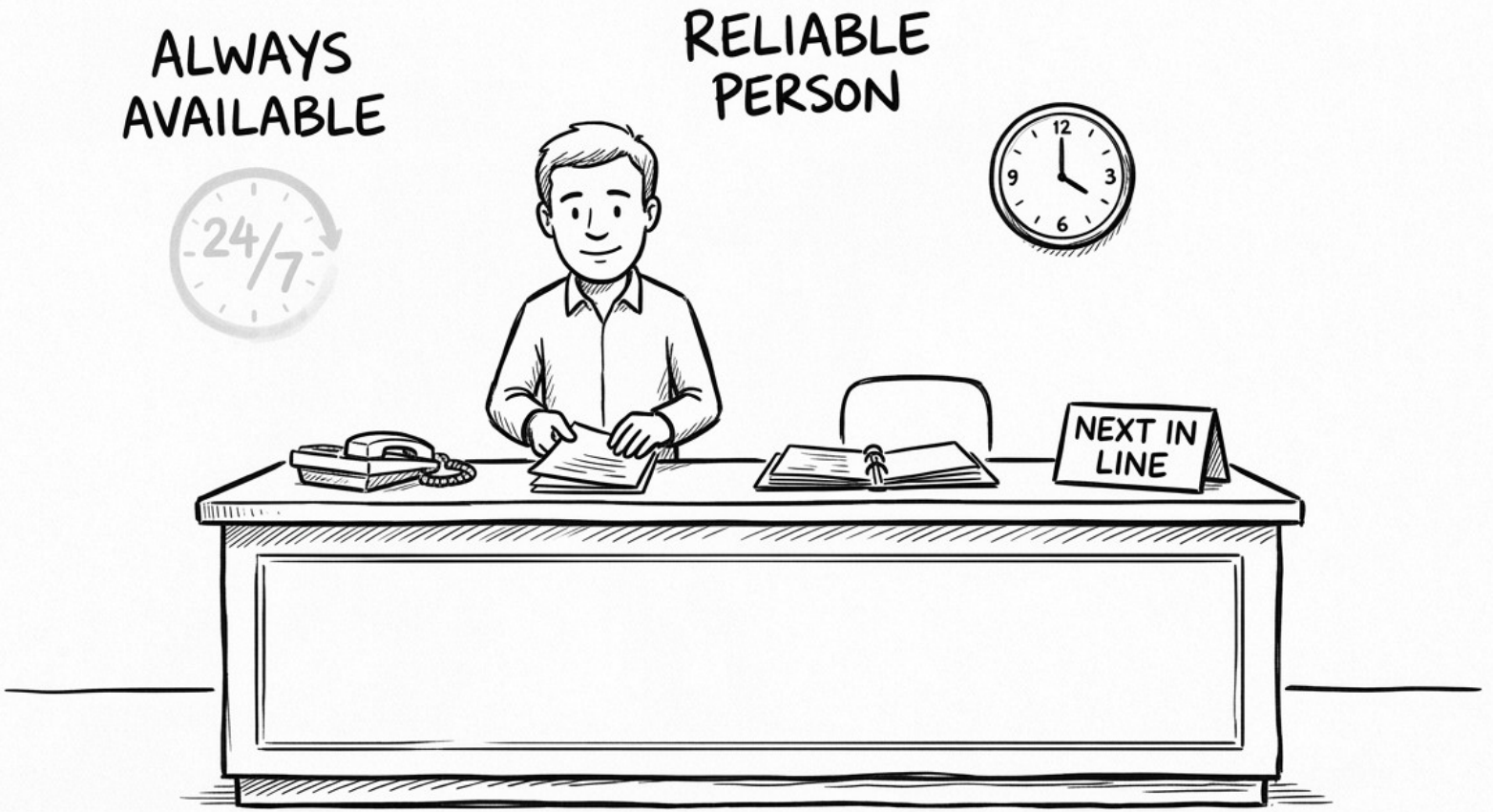
What is Resilience?




EXTERNAL
PRESSURE/CHALLENGE
RESILIENT
PERSON

What is Reliability?




ALWAYS AVAILABLE
24/7
RELIABLE PERSON
12
3
6
9
NEXT IN LINE

Two Different Goals. One Broken Assumption.

RELIABILITY

Keep the system running

- Uptime metrics (99.9%, 99.99%)
- Redundancy & failover
- Monitoring & alerting
- SLIs, SLOs, error budgets
- Assumes failures are rare

VS

RESILIENCE

Recover fast when it does

- Graceful degradation under failure
- Fast detection & self-healing
- Blast-radius control
- Validated failure hypothesis
- Failures are expected -- prepare

Key insight: Reliability tells you IF something failed. Resilience determines what happened NEXT.

PART 1 · RELIABILITY VS RESILIENCE

Your System "Works", Until It Doesn't.

76%
of outages caused by unknown unknowns

45 min
average MTTR for a P1 in microservices architectures

$80K+
annual exposure for a 10-service app with zero coverage

7 in 10
services with ZERO chaos or load test coverage

Sources: Harness Passive Risk Detection data · Gremlin State of Chaos Engineering 2024

What Is Chaos Engineering?

"The discipline of experimenting on a system to build confidence in its capability to withstand turbulent conditions."

1. **Define Steady State**
   What does 'normal' look like?
2. **Hypothesise**
   What do we believe happens when X fails?
3. **Inject Failure**
   Kill a pod, drop packets, spike CPU
4. **Observe & Learn**
   Was the hypothesis confirmed or busted?

Chaos is NOT about breaking things.

It's about finding weaknesses before production does.

Every steady-state hypothesis you confirm is a failure your users never experience.

The Cost of Not Doing Chaos Engineering

MYTH 1

“We have 99.9% uptime so we're fine”

REALITY

99.9% uptime = 8.7 hrs downtime/year. Distributed systems fail in correlated bursts -- your SLO won't warn you.

MYTH 2

“We test in staging, production is safe”

REALITY

Staging doesn't have real traffic, prod data volumes, or the unpredictable timing of real failures. It's necessary but not sufficient.

MYTH 3

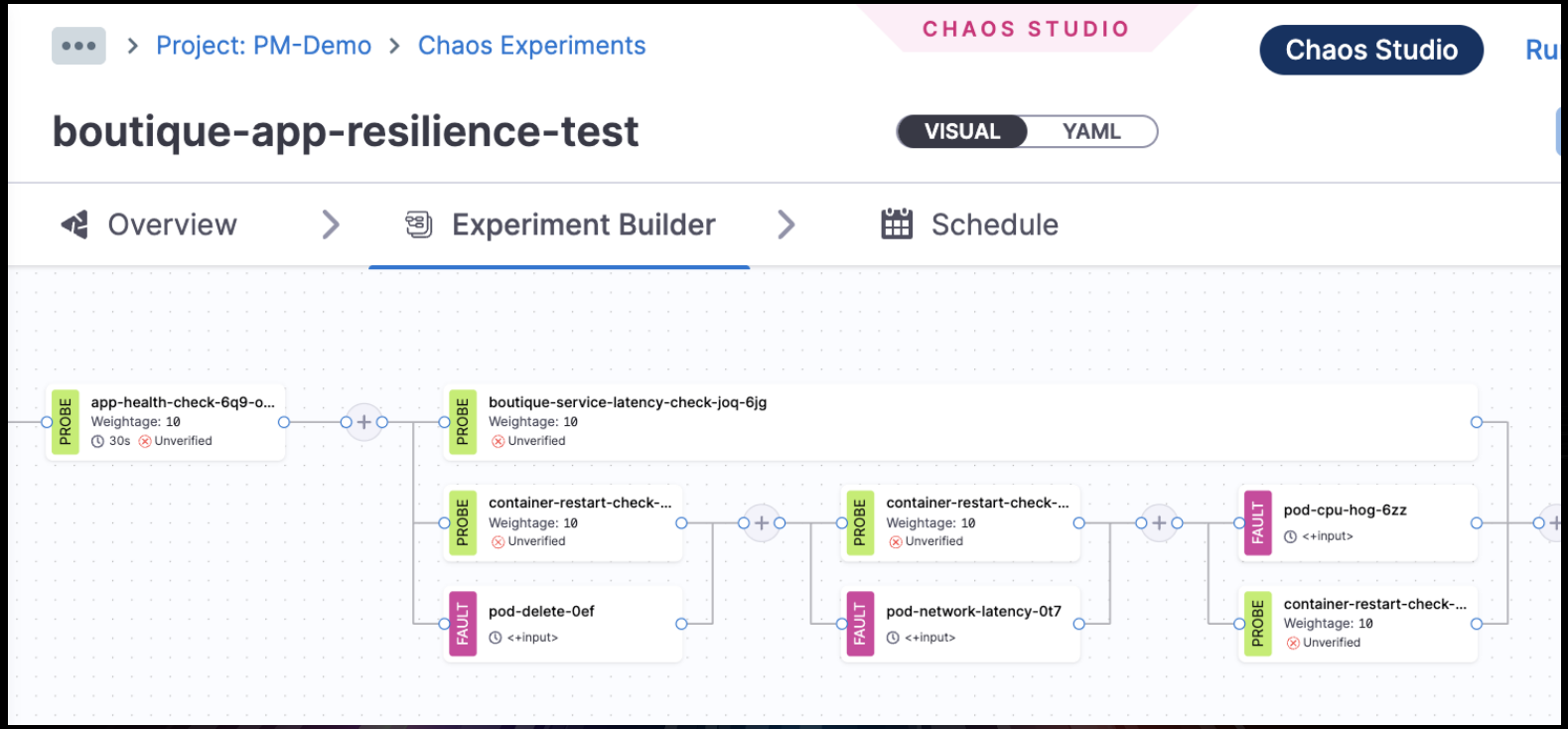
“Our engineers know how the system behaves”

REALITY

Every deployment silently invalidates past assumptions. The system your team knows today is not the system running in prod tonight.

Chaos Engineering is not optional risk -- it is the insurance policy that makes reliability promises real.

Anatomy of a Chaos Experiment



PART 2



The SRE's Role in Chaos Engineering

Shift Left

Deploy Gate

Production

PART 2 : SRES & CHAOS ENGINEERING

Shift Left: Find Weaknesses Before Production Does

1. **Define Steady State Hypothesis**
   For every critical service, specify the observable normal -- latency p99, error rate, throughput.

2. **Identify Blast Radius**
   Map which services cascade if this one fails. Build the dependency graph.

3. **Run Chaos in Staging**
   Pod-delete, network partition, CPU throttle -- before the code ships.

4. **Set Resilience Quality Gates**
   Block promotion if Steady State Hypothesis fails. Resilience = code quality.

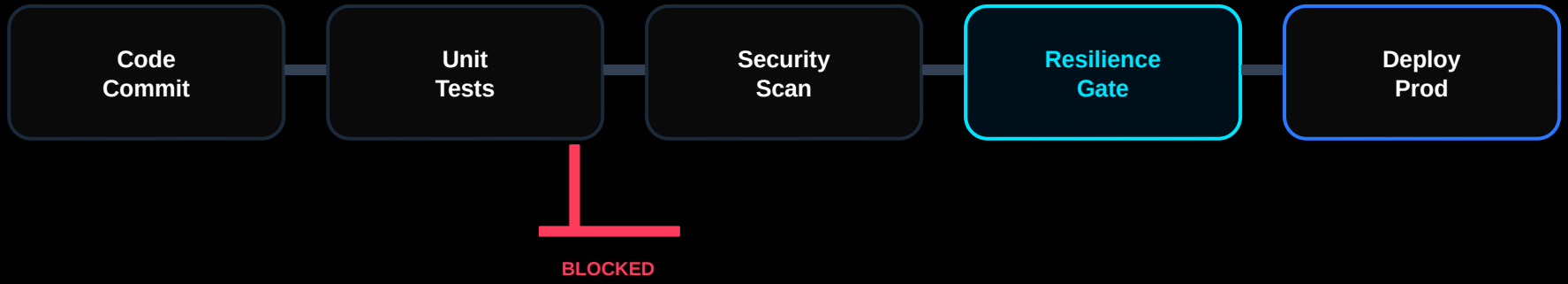
Passive Risk Detection

Zero-config scan of CD metadata, K8s manifests & pipeline history

FAIL	Single-replica service -- no PDB
FAIL	Missing resource limits & requests
WARN	No circuit breaker on external dep
WARN	Zero test coverage for 7 services
INFO	No liveness probe configured

PART 2 · SRES & CHAOS ENGINEERING

Deployment Gate: Resilience Is a Quality Signal



What the Resilience Gate Checks

- Steady State Hypothesis confirmed in staging
- Resilience Risk Score below gate threshold (< 100)
- Payment service: single replica, no PDB -- FAIL
- Cart service: zero chaos test coverage -- FAIL

Resilience Risk Score

FAIL x 10 + WARN x 4 + INFO x 1

- Score > 100 -> block deployment
- Score 50-100 -> warn + notify
- Score < 50 -> green light

PART 2 · SRES & CHAOS ENGINEERING

After Deployment: Chaos Doesn't Stop at Merge

Traffic is real -- and unpredictable

Staging never replicates prod load, seasonal spikes, or third-party latency.

Config drift silently invalidates safety

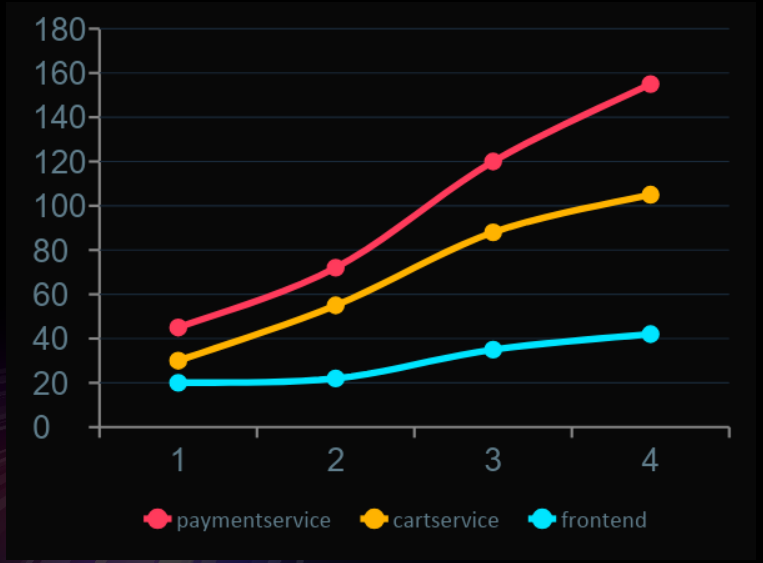
A settings change, a new dep, a quota update -- any can break a confirmed hypothesis.

Game Days build muscle memory

Scheduled, controlled production experiments keep the team sharp on incident response.

Post-incident chaos is diagnostic

After every P1 closes, run a targeted experiment to verify the structural cause is fixed.



Score above 100 = deployment should be blocked

The SRE Chaos Maturity Journey

L1 Ad-hoc — Manual experiments, tribal knowledge, no repeatable process. Chaos happens TO you.

L2 Structured — Defined SSH, blast-radius controls, staging experiments. Chaos is a practice.

L3 Integrated — Resilience gates in CI/CD. Every deployment validated. SLO-linked experiments.

L4 Continuous — Automated Game Days, production experiments on schedule, RRS tracked per pipeline.

L5 Predictive — AI surfaces risks before experiments run. Passive detection plus auto-generation.

Where is your team today? L4-L5 is where Harness Resilience Testing plays.

PART 3



Chaos Engineering in the AI Era

Find risks without running experiments

Generate experiments from natural language

Autonomous post-run analysis & learning

AI can help scale Chaos Engineering

Traditional Approach

- 1 or 2 SREs design every experiment manually
- Service knowledge lives in people's heads
- Months between experiment design and execution
- Post-mortems happen after the outage, not before
- Coverage measured in “we think we covered the main paths”

Average: 2-4 hours per experiment designed

AI-Augmented Approach

Risk Discovery
Zero-config passive scan surfaces gaps without running a single experiment

AI Experiment Design
Natural language -> fully parameterised chaos/load test in seconds

Auto-Generate from Risks
One click converts a detected FAIL risk into a runnable experiment

Instant Post-Run Analysis
Pass/fail verdict, root cause hypothesis, next-step recommendations

Passive Risk Detection: Find Failures Without Causing Them

CD Pipeline Executes
Harness CD metadata

K8s Manifest Scanned
Replicas, probes, limits

APM & Incidents Read
Past P1s, error rates

Risk Engine Runs
350+ rules, topology-aware

PRD Report Generated
FAIL · WARN · INFO + RRS

FAIL **paymentservice** **+10**
Single replica -- zero PDB. Full outage on pod eviction.

FAIL **cartservice** **+10**
No readiness probe. Traffic hits pod before it is ready.

WARN **checkoutservice** **+4**
External payment dep has no retry/circuit breaker.

WARN **frontend** **+4**
No load test coverage -- performance baseline unknown.

Total RRS: 192 | Gate threshold: 100 | Deployment BLOCKED

AI-Powered Chaos: From Risk to Runnable in Seconds

BEFORE

Manually Designed

1. SRE reads incident history
2. Maps service topology manually
3. Writes experiment YAML
4. Configures blast radius
5. Sets up rollback conditions

Average: 2-4 hours per experiment designed

AFTER

AI-Generated

Natural language input:

"*What happens to checkout if Redis fails?*"

AI generates complete experiment config:

```
target: cartservice
namespace: default  blast_radius: 100%
fault: pod-delete  duration: 5min
steady_state: HTTP 200  p99 < 500ms
```

Time to runnable experiment: under 30 seconds

Post-Run: AI delivers instant verdict | anomaly detection | root cause hypothesis | next-experiment recommendations

The Resilience-First Future

THEN

React to Failures

- Outages happen -> post-mortem
- Chaos = optional, scary
- SREs run manual GameDays once a year
- No idea if staging reflects prod

NOW

Shift Left on Resilience

- Chaos in CI/CD pipelines
- Resilience gates block bad deploys
- Passive risk discovery on every push
- RRS score as a quality metric

NEXT

Autonomous Resilience

- AI detects risk before humans do
- Experiments auto-generated from topology
- Self-healing with AI recommendations
- Resilience as code -- version-controlled

Start with passive risk detection. You'll find failures you didn't know existed -- before your users do.

Harness Resilience Testing

Know your weaknesses before your customers do.

Chaos Testing

- Inject controlled faults
- Identify weak points
- Validate system behavior under stress
- Proactive failure simulation

Load Testing

- Simulate high user traffic
- Measure performance limits
- Ensure scalability & stability
- Identify bottlenecks

DR Testing

- Verify backup & restore procedures
- Test failover mechanisms
- Ensure business continuity
- Minimize downtime

Thank You

